Word Ladder: Shortest Ladders with BFS on a Word Graph

CS314 · Analysis of Algorithms · Mini Project (Graphs: BFS)

1. Problem

Given a start word, a target word and a dictionary, turn the start word into the target by changing one letter at a time so that every intermediate word is in the dictionary, using as few changes as possible. For hit → cog with the words hot, dot, dog, lot, log, cog, the shortest ladder is hit, hot, dot, dog, cog: 5 words, 4 changes. The program returns the ladder, or reports that none exists. As an extension, it finds the longest shortest ladder in the whole dictionary.

2. Graph Model

Words of the same length are the vertices of an undirected, unweighted graph. Two words are joined by an edge when they differ in exactly one letter, so a ladder is a path from the start vertex to the target vertex, and the fewest changes means the fewest edges. Comparing every pair of words costs $\Theta(N^2 \cdot L)$. Instead, each word of length L is put into L wildcard buckets, for example `hit` goes to `*it`, `h*t` and `hi*`. Two words differ in exactly one position if and only if they share a bucket, and then they share exactly one, so every edge is produced once from the bucket of its differing position. The figure shows the example graph with the shortest ladder in red.

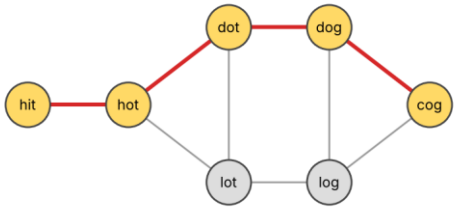


Figure 1. Word graph of the example

3. Algorithm and Complexity

The program follows the four milestones. (1) Load the word list and group the words by length. (2) Build the adjacency list with wildcard buckets. (3) Run BFS from the start word, keeping the distance `dist` and the parent `parent` of every vertex; a vertex is marked when it is enqueued, so it enters the queue once, and the search stops when the target is dequeued. (4) Follow the parent pointers back from the target and reverse them to print the ladder.

```
def bfs(adj, s, target=None):            def path(parent, v):
    dist = {s: 0}; parent = {s: None}        p = []
    q = deque([s])                           while v is not None:
    while q:                                     p.append(v); v = parent[v]
        u = q.popleft()                      return p[::-1]
        if u == target: break
        for v in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + 1; parent[v] = u; q.append(v)
    return dist, parent
```

Correctness. BFS removes vertices from the queue in non-decreasing order of distance, so the first time it reaches a word is along a path with the fewest edges, and the parent pointers spell out that path. If the target is never reached, no ladder exists.

Complexity. Let N be the number of words of length L, V the number of vertices and E the number of edges. Forming the wildcard keys costs O(L) per bucket entry, so $O(N \cdot L^2)$ overall. A bucket with k words adds $k(k-1)$ adjacency entries, and every edge lies in exactly one bucket, so all buckets together add exactly 2E entries. Building the graph is therefore $O(N \cdot L^2 + E)$, against $\Theta(N^2 \cdot L)$ for comparing all pairs. BFS scans each adjacency list once, so it takes O(V + E) time and O(V) space, and stopping at the target does not change the worst case. The extension runs BFS from every word, which is $O(V \cdot (V + E))$ per word length.

Step	Time	Space
Group words by length	$O(N \cdot L)$	$O(N \cdot L)$
Build graph with wildcard buckets	$O(N \cdot L^2 + E)$	$O(N \cdot L + E)$
Build graph comparing all pairs (naive)	$\Theta(N^2 \cdot L)$	O(N + E)
BFS with parent pointers	O(V + E)	O(V)
Rebuild the ladder with path()	O(ladder length)	O(ladder length)
Extension: BFS from every word	$O(V \cdot (V + E))$	O(V)

4. Experiments

Setup: Python 3.12, time.perf_counter, best of several runs. The word list has 61,935 lowercase words (base forms of the Hunspell en_US dictionary, which include some abbreviations). For scaling, random dictionaries of $N = 10^3$ to 10^5 words over the alphabet a–d were used, with the word length chosen so that the dictionary fills about a quarter or less of all strings; BFS started at the word with the most neighbours. Table 1 shows the results.

N	V	E	Build (ms)	BFS (ms)	BFS ns / (V+E)
1,000	1,000	2,183	2.0	0.20	63
2,000	2,000	2,667	5.5	0.35	75
5,000	5,000	4,560	18.3	0.79	83
10,000	10,000	18,433	51.4	4.12	145
20,000	20,000	20,543	179.4	7.77	192
50,000	50,000	128,789	501.7	38.97	218
100,000	100,000	142,768	1530.3	77.81	321

Table 1. Graph build and BFS time on random dictionaries

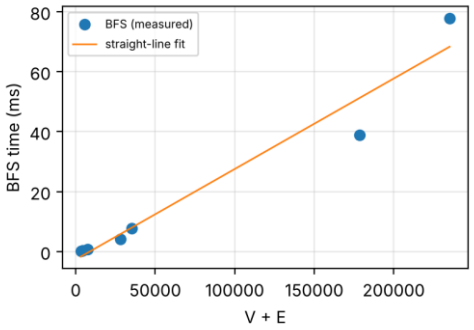


Figure 2. Time against V + E

Observations. BFS time grows in proportion to V + E: a straight-line fit through the seven runs has $R^2 = 0.95$ and a slope of about 300 ns per vertex or edge. The cost per unit rises from 63 ns to 321 ns as the graph grows 100 times, probably because the large dictionaries no longer fit in the processor cache; this was not tested separately. Table 2 compares the two ways of building the graph on random samples of the 4,389 five-letter words; both give identical graphs, and the wildcard buckets are 29 to 291 times faster, with the gap widening as N grows, as $\Theta(N^2 \cdot L)$ against $O(N \cdot L^2 + E)$ predicts.

Words N	Edges E	Buckets (ms)	All pairs (ms)	Speed-up
250	17	0.42	12	29×
500	95	0.84	54	64×
1,000	348	2.13	236	110×
2,000	1,167	4.73	974	206×
4,000	4,904	12.39	3600	291×

Table 2. Building the graph for five-letter words

On the full word list, one query takes 7 to 15 ms including grouping, graph building and BFS. Because the full list contains words such as cit and cot, hit → cog needs only 3 changes (hit, cit, cot, cog), shorter than the 4 changes of the 6-word example. The longest shortest ladder in the dictionary has 38 changes: amigo → radii for five-letter words, with decani → haslet equally long for six-letter words; for four-letter words it is 22 (amen → inti). Running BFS from every word of every length took 7.2 s. The code is checked by 22 unit tests, including a comparison with Floyd–Warshall distances on random dictionaries.

5. Conclusion

Modelling words as vertices and one-letter changes as edges turns the puzzle into an unweighted shortest-path problem, which BFS solves in O(V + E) with parent pointers giving the ladder. The wildcard buckets make the graph construction practical: they replaced a quadratic comparison and were up to 291 times faster in the experiments. The measured BFS time is consistent with linear growth in V + E, with a somewhat rising cost per item on the largest graphs. The main limits are the quality of the word list, which decides which ladders exist, and the fact that BFS is only correct while all changes cost the same.